UNIT-3 FILE HANDLING IN PYTHON

Q1) What are Files in Python?

Ans:Files are used to store information permanently on a computer.

- They save data in storage devices like a hard disk or SSD.
- RAM memory loses data when the computer is turned off, but files keep the data safe.
- We use files when we want to save data for future use.

Steps to work with a file in Python

- First, open the file.
- Then, read data from it or write data into it.
- Finally, close the file after finishing the work.
- Closing the file frees the computer resources being used by the file.

Types of operations on files

- Open a file
- Read from a file
- Write into a file
- Close the file

Q2) Explain two types of files.

Ans:

Types of Files in Python

There are mainly **2 types of files** in Python:

- **Text Files**
- **Binary Files**

1. Binary Files

- Binary files store data in the form of **0s and 1s** (binary format).
- Humans cannot easily read binary files directly.
- A normal text editor cannot properly show their content.
- Computers understand these files easily.

Important Point

- To open binary files, we usually need special software.

Examples

- .doc → Microsoft Word files
- .pdf → PDF files
- .jpg, .png → Image files
- .mp4 → Video files
- .mp3 → Audio files

Common Binary Files

- **Document files:** .pdf, .doc, .xls

- **Image files:** .png, .jpg, .gif
- **Video files:** .mp4, .avi, .mkv
- **Audio files:** .mp3, .wav
- **Database files:** .sqlite, .mdb
- **Compressed files:** .zip, .rar
- **Executable files:** .exe, .dll

Simple Understanding

- Binary files are mainly meant for computers and software programs.

2. Text Files

- Text files store data in the form of **characters and words**.
- Humans can easily read them.
- They can be opened using a normal text editor like Notepad.

Important Points

- A text file contains lines of text.
- Each line ends with a special character called **newline (\n)**.
- Python mainly uses **ASCII** and also supports **Unicode** characters.

Examples

- .txt
- .html

- .css
- .json
- .py
- .java
- .csv

Common Text Files

- **Web files:** .html, .xml, .css, .json
- **Programming files:** .py, .java, .c, .js
- **Document files:** .txt, .rtf
- **Table data files:** .csv
- **Configuration files:** .ini, .cfg

Simple Understanding

- Text files are easy for humans to read and edit.

Difference Between Text Files and Binary Files

Text Files	Binary Files
Human readable	Not human readable
Can open in Notepad	Need special software

Stores text/characters	Stores data in binary form
Easier to edit	Harder to edit manually
Example: .txt, .py	Example: .pdf, .jpg

Q3) How to open or create a new file?

Ans:

Opening or Creating a File in Python

- In Python, we use the open() function to open an existing file or create a new file.
- If the file does not exist, Python can create it.
- If you do not give the full folder path, the file will be created in the same folder where the Python program is saved.

Syntax of open()

file_object = open(file_name, access_mode, buffering)

Meaning of each part

- file_object → Variable that stores the opened file.
- file_name → Name of the file.
- access_mode → Tells Python how to use the file (read, write, append, etc.).
- buffering → Controls temporary memory storage while loading the file.

What does open() return?

- open() returns a **file object** (also called file handle).
- We store it in a variable.

Example

```
f = open("test.txt", "r")
```

Here:

- f is the file object.
- "test.txt" is the file name.
- "r" means read mode.

Access Modes in Python

1. Read Mode (r)

- Used only for reading the file.

- File pointer starts from the beginning.
- Cannot modify the file.

Example

```
f = open("test.txt", "r")
```

2. Write Mode (w)

- Used to write data into the file.
- If the file already has data, old data will be erased (overwritten).

Example

```
f = open("test.txt", "w")
```

3. Append Mode (a)

- Used to add new data at the end of the file.
- Old data is not removed.

Example

```
f = open("test.txt", "a")
```

Modes for Reading and Writing Together

r+

- Used for both reading and writing.

```
f = open("test.txt", "r+")
```

a+

- Used for reading and appending data.

```
f = open("test.txt", "a+")
```

Binary File Modes

Mode	Meaning
rb	Read binary file
wb	Write binary file
ab	Append binary file
rb+	Read and write binary file
ab+	Append and read binary file

What is Buffering?

- Buffering means temporarily storing file data in memory before fully loading or saving it.

- It helps improve speed.

Buffering values

- 0 → Buffering OFF
- 1 → Buffering ON

Examples of Opening Files

Open file from current folder

```
f = open("test.txt", "r")
```

Open file using full path

Raw string method

```
f = open(r"D:\temp\data.txt", "r")
```

Absolute path method

```
f = open("D:\\temp\\data.txt", "r")
```

Simple Summary

Mode	Purpose
r	Read file
w	Write file

a	Add data at end
r+	Read + Write
a+	Read + Append

Q4) How to close a file?

Ans: Closing Files in Python

- After finishing work with a file, we should close it.
- Closing a file saves the changes properly.
- It also frees the computer resources used by the file.
- In Python, we use the close() method to close a file.

Syntax

```
file_object.close()
```

Example

```
string = "This is a String in Python"

my_file = open("my_file_name.txt", "w+", 1)
```

```
my_file.write(string)

my_file.close()

print(my_file.closed)
```

Explanation of the Program

Step 1

```
string = "This is a String in Python"
```

- A string is stored in the variable string.

Step 2

```
my_file = open("my_file_name.txt", "w+", 1)
```

- Opens the file named "my_file_name.txt".
- "w+" means read and write mode.
- 1 means buffering is ON.

Step 3

```
my_file.write(string)
```

- Writes the string into the file.

Step 4

my_file.close()

- Closes the file after writing.

Step 5

print(my_file.closed)

- Checks whether the file is closed or not.
- It prints:
  - True → File is closed
  - False → File is still open

Simple Summary

Method	Purpose
open()	Open/Create a file
write()	Write data into file
close()	Close the file
closed	Check file status

Q5) How to read information in a file?

Ans:

Reading Information from a File in Python

- To read data from a file, the file must be opened in **read mode (r)**.
- Python provides different methods to read files.

Methods Used to Read Files

Method	Purpose
read()	Reads the whole file or a specific number of characters
readline()	Reads one line at a time
readlines()	Reads all lines and stores them in a list

1. read([n])

- Reads data from the file.
- If n is given, it reads only n characters.

- If n is not given, it reads the entire file.

Example 1: Read First 5 Characters

```
my_file = open("C:/Documents/Python/test.txt", "r")

print(my_file.read(5))
```

Output

```
Hello
```

Explanation

- read(5) reads only the first 5 characters from the file.

Example 2: Read Entire File

```
my_file = open("C:/Documents/Python/test.txt", "r")

print(my_file.read())
```

Output

```
Hello World
Hello Python
Good Morning
```

Explanation

- Since no number is given inside read(), Python reads the complete file.

2. readline([n])

- Reads only one line at a time.
- If n is given, it reads only n characters from that line.

Example 3: Read First 2 Characters of a Line

```
my_file = open("C:/Documents/Python/test.txt", "r")

print(my_file.readline(2))
```

Output

```
He
```

Explanation

- readline(2) reads the first 2 characters of the line.

Example 4: Read One Complete Line

```
my_file = open("C:/Documents/Python/test.txt", "r")
```

```
print(my_file.readline())
```

Output

```
Hello World
```

Explanation

- readline() reads one full line from the file.

3. readlines()

- Reads all lines from the file.
- Stores them inside a list.

Example 5: Read All Lines

```
my_file = open("C:/Documents/Python/test.txt", "r")

print(my_file.readlines())
```

Output

```
['Hello World\n', 'Hello Python\n', 'Good Morning']
```

Explanation

- Each line becomes an item in the list.

- \n represents a newline character.

Reading a Specific Line from a File

Example 6: Read Only the 4th Line

```
line_number = 4

fo = open("C:/Documents/Python/test.txt", "r")

currentline = 1

for line in fo:
    if(currentline == line_number):
        print(line)
        break

    currentline = currentline + 1
```

Output

How are You

Explanation

- A for loop checks each line one by one.

- When the line number becomes 4, it prints that line.

Reading the Entire File at Once

Example 7

```
filename = "C:/Documents/Python/test.txt"

filehandle = open(filename, "r")

filedata = filehandle.read()

print(filedata)
```

Output

```
Hello World
Hello Python
Good Morning
How are You
```

Explanation

- read() reads the complete content of the file at once.

Simple Summary

Function	What it Does
read(5)	Reads first 5 characters
read()	Reads full file
readline()	Reads one line
readlines()	Reads all lines as a list

Q6) How to write and append data in a file.

Ans:

Writing Data into a File in Python

- To write data into a file, we must open the file in write mode (w).
- Writing mode removes the old content and replaces it with new content.
- So, be careful because previous data can be lost.

Methods Used for Writing

Method	Purpose

write()	Writes a string into the file
writelines()	Writes multiple lines from a list

1. write(string)

- Used to write text into a file.

Example 1: Write a Single Line

```
my_file = open("C:/Documents/Python/test.txt", "w")

my_file.write("Hello World")
```

Explanation

- Opens the file in write mode.
- Writes "Hello World" into the file.
- Old content will be erased.

Writing Multiple Lines

Example 2

```
my_file = open("C:/Documents/Python/test.txt", "w")
```

```
my_file.write("Hello World\n")

my_file.write("Hello Python")
```

Explanation

- \n moves the cursor to the next line.
- Output inside the file will be:

Hello World
Hello Python

Without \n

If we do not use \n, the output becomes:

Hello WorldHello Python

2. writelines(list)

- Used to write multiple lines together from a list.

Example 3

```
fruits = ["Apple\n", "Orange\n", "Grapes\n", "Watermelon"]

my_file = open("C:/Documents/Python/test.txt", "w")
```

```
my_file.writelines(fruits)
```

Explanation

- Each item in the list is written into the file.
- \n creates new lines.

Output

Apple
Orange
Grapes
Watermelon

Appending Data into a File

- Appending means adding new data at the end of the file.
- Old content is not deleted.
- For appending, we use a+ mode.

Example 1: Append Text

```
my_file = open("C:/Documents/Python/test.txt", "a+")

my_file.write("Strawberry")
```

Explanation

- Adds "Strawberry" at the end of the file.

Example 2: Append in New Line

```
my_file = open("C:/Documents/Python/test.txt", "a+")

my_file.write("\nGuava")
```

Explanation

- \n moves to the next line before writing.
- "Guava" will appear on a new line.

Difference Between Write and Append

Write Mode (w)	Append Mode (a+)
Removes old data	Keeps old data
Writes from beginning	Adds data at the end
Overwrites file	Appends file

Simple Summary

Function	Purpose
write()	Write one string
writelines()	Write multiple lines
w	Write mode
a+	Append and read mode
\n	Move to next line

Q7) Explain about flush() function.

Ans:

flush() Function in Python

- When we write data into a file, Python does not immediately save it to the actual file.
- First, the data is stored in a temporary memory called a **buffer**.

- Later, Python transfers the buffered data into the real file.

What does flush() do?

- flush() forces Python to immediately write the buffered data into the file.
- It saves the current data without closing the file.

Important Point

- When we use close(), Python automatically calls flush().
- But if we want to save data before closing the file, we use flush() manually.

Syntax

file_object.flush()

Without flush()

Example

```
f = open("temp.txt", "w")

f.write("Hello World")

input("Press any key...")
```

```
f.close()
```

What Happens?

- "Hello World" is stored only in buffer memory.
- Before pressing a key:
  - If you open temp.txt, it may appear empty.
- After close():
  - Data is finally written into the file.

With flush()

Example

```
f = open("temp.txt", "w")

f.write("Hello World")

f.flush()

input("Press any key...")

f.close()
```

What Happens?

- flush() immediately writes "Hello World" into the file.

- Even before pressing a key, the content is already saved in temp.txt.
- close() later saves any remaining buffered data.

Difference Between flush() and close()

flush()	close()
Saves buffered data immediately	Saves data and closes file
File remains open	File becomes closed
Can continue working with file	Cannot use file again unless reopened

Simple Summary

- Python temporarily stores file data in a buffer.
- flush() sends buffer data to the actual file immediately.
- close() automatically performs flush() and then closes the file.

Q8) How to remove whitespaces after reading from a file?

Ans:

Removing Whitespaces After Reading from a File

- Functions like read(), readline(), and readlines() read data from a file.
- While reading, they also read:
  - Spaces
  - Tabs
  - Newline characters (\n)

Sometimes we do not want these extra characters.
For removing them, Python provides:

- strip()
- lstrip()
- rstrip()

1. strip()

- Removes spaces or characters from **both sides** (left and right).

Example

```
text = "   Hello Python   "

print(text.strip())
```

Output

Hello Python

2. lstrip()

- Removes spaces or characters only from the **left side**.

Example

```
text = "   Hello Python   "

print(text.lstrip())
```

Output

Hello Python

3. rstrip()

- Removes spaces or characters only from the **right side**.

Example

```
text = "   Hello Python   "

print(text.rstrip())
```

Output

Hello Python

Removing Newline Characters from File Data

Example

```
my_file = open("test.txt", "r")

line = my_file.readline()

print(line.strip())
```

Explanation

- readline() reads the line including \n.
- strip() removes the newline character and extra spaces.

Simple Difference

Function	Removes From
strip()	Both left and right

lstrip()	Left side only
rstrip()	Right side only

Simple Summary

- File reading functions may include unwanted spaces and \n.
- strip() functions help clean the text.
- These functions are very useful when working with file data.

Q9) Explain about File Pointer.

Ans:

File Pointer in Python

- Every file in Python has a **file pointer**.
- A file pointer keeps track of the current position inside the file.
- Reading and writing operations happen at the position of the file pointer.

What Happens During Read/Write?

When we read or write data:

1. Operation happens at the current file pointer position.
2. The file pointer automatically moves forward.

Simple Example

Imagine a file contains:

Hello

If the file pointer is at the beginning:

H e l l o
^
Pointer

Example Program

```
myfile = open("ipl.txt", "r")

ch = myfile.read(1)

print(ch)
```

Explanation

Step 1

myfile = open("ipl.txt", "r")

- Opens the file in read mode.
- File pointer starts at the first character.

Step 2

ch = myfile.read(1)

- Reads only 1 character.
- Suppose the first character is H.
- H is stored in ch.

Step 3

After reading:

H e l l o
^

Pointer moved

- The first character is consumed.
- File pointer moves to the next character automatically.

Important Point

- Every time you read data, the pointer moves forward.
- The next read operation starts from the new position.

Simple Summary

Term	Meaning
File Pointer	Current position inside file
read(1)	Reads 1 character
Pointer Movement	Automatically moves after reading/writing

Q10) Explain about Binary File Operations in Python.

Ans:

Binary File Operations in Python

In Python, we mainly perform **4 operations** on binary files:

1. Insert/Appending records
2. Reading records
3. Searching records
4. Updating records

1. Inserting/Appending Records in a Binary File

- To store data in a binary file, Python uses the pickle module.
- The dump() function is used to write objects into the binary file.

Steps

- Import the pickle module.
- Open the file in binary write or append mode.
- Use pickle.dump() to store data.

Example

```
import pickle

f = open("student.dat", "ab")

student = [101, "Akhil", 95]

pickle.dump(student, f)

f.close()
```

Explanation

- "ab" → Append binary mode.
- pickle.dump() stores the record into the binary file.

2. Reading Records from a Binary File

- pickle.load() is used to read data from a binary file.

Syntax

object = pickle.load(file)

Example

```
import pickle

f = open("student.dat", "rb")

data = pickle.load(f)

print(data)

f.close()
```

Explanation

- "rb" → Read binary mode.
- load() reads the stored object from the file.

3. Searching a Record in a Binary File

- Searching is usually done using a unique value like:
  - Roll number

- ID
- Name

Steps

- Open file in read binary mode.
- Read records one by one using load().
- Compare the record value with the search value.
- If matched, display the record.

Example

```
import pickle

f = open("student.dat", "rb")

roll = int(input("Enter roll number: "))

found = False

try:
    while True:
        data = pickle.load(f)

        if data[0] == roll:
         print(data)
         found = True
```

```
        break

except EOFError:
    if found == False:
        print("Record not found")

f.close()
```

Explanation

- found checks whether the record exists or not.
- EOFError occurs when the file ends.

4. Updating a Record in a Binary File

- Updating means changing existing data in the file.

Steps

1. Search for the record.
2. Modify the required value.
3. Move file pointer if needed using seek().
4. Save updated data.

What is seek()?

- seek() moves the file pointer to a specific position.

- Used for random access in files.

Example

f.seek(0)

Explanation

- Moves the file pointer to the beginning of the file.

Simple Summary

Operation	Function Used
Insert record	pickle.dump()
Read record	pickle.load()
Search record	Compare values while reading
Update record	Modify data + seek()

Important Functions

Function	Purpose

dump()	Write object to binary file
load()	Read object from binary file
seek()	Move file pointer
pickle	Module used for binary files

Q11) Explain Random Access in Files using tell() and seek().
Ans:

Random Access in Files using tell() and seek()

- Until now, we read files one line after another (sequential access).
- Python also allows **random access**, which means:
  - We can move directly to any position in the file.
- For this, Python uses:
  - seek()
  - tell()

1. seek() Function

- seek() changes the position of the file pointer.
- The file pointer acts like a cursor showing where reading or writing will happen.

Syntax

f.seek(offset, from_what)

Meaning of Parameters

Parameter	Meaning
offset	Number of bytes to move
from_what	Starting reference position

Values of from_what

Value	Meaning
0	Beginning of file (default)

1 Current position

2 End of file

Absolute Positioning

- Moves pointer to an exact byte position.

Syntax

f.seek(file_location)

Example

f.seek(20)

Explanation

- Moves the file pointer directly to the 20th byte.
- It does not matter where the pointer was before.

Relative Positioning

- Moves pointer forward or backward relative to current position.

Examples

Move 10 bytes forward from current position

f.seek(10, 1)

Move 10 bytes backward

f.seek(-10, 1)

Move 20 bytes backward

f.seek(-20, 1)

Move 10 bytes from beginning

f.seek(10, 0)

2. tell() Function

- tell() returns the current position of the file pointer.

Syntax

f.tell()

Example

f = open("test.txt", "r")

print(f.tell())

Output

0

Explanation

- When a file is opened in read or write mode, the pointer starts at byte 0.

Another Example

```
f = open("test.txt", "r")

f.read(5)

print(f.tell())
```

Output

5

Explanation

- After reading 5 characters, the pointer moves to position 5.

Important Points

- In **read/write mode**, file pointer starts at beginning (0).

- In **append mode**, pointer starts at the end of file.

Difference Between seek() and tell()

Function	Purpose
seek()	Move file pointer
tell()	Show current pointer position

Simple Summary

- seek() changes pointer position.
- tell() tells current pointer position.
- These functions help in random access of files.

Q12) Explain about CSV files.

Ans:

CSV File Operations in Python

What is a CSV File?

- CSV stands for **Comma Separated Values**.
- It is a simple text file used to store tabular data.

- Data is arranged in rows and columns like a table.

Important Features of CSV Files

- CSV files are human readable.
- They store data in plain text format.
- Each line in the file represents one record (row).
- Values in a row are separated using commas.

Example of a CSV File

Name,Age,City
Akhil,20,Hyderabad
Ravi,22,Delhi

Explanation

Row	Meaning
First row	Column names (headers)
Other rows	Actual data

What is a Delimiter?

- A delimiter separates values in a file.
- In CSV files, comma , is commonly used.

Other Delimiters

Delimiter	Symbol
Comma	,
Tab	\t
Colon	:
Semicolon	;

Why CSV Files are Used

- Easy to create and read.
- Used to store spreadsheet and database data.
- Supported by many software applications.

Programs That Use CSV Files

CSV files can be opened in:

- Microsoft Excel
- OpenOffice Calc
- Google Sheets
- Database software
- Python programs

Real-Life Example

A student CSV file may look like:

RollNo,Name,Marks
101,Akhil,95
102,Ravi,88
103,Anu,91

Important Points

- CSV files store only text data.
- They are lightweight and simple.
- Very useful for data exchange between programs.

Simple Summary

Term	Meaning

CSV	Comma Separated Values
Record	One row of data
Field	One value in a row
Delimiter	Character used to separate values

Q14) Why CSV files are useful.

Ans:

Why CSV Files are Useful

- Social networking sites and apps generate huge amounts of data.
- This data is often unorganized and difficult to manage.
- CSV files help organize this large data in a structured tabular format.

How CSV Helps

- CSV arranges data into rows and columns.
- This makes the data easy to store, read, and manage.

- Since CSV files are plain text files, developers can easily use them in applications and websites.

Advantages of CSV Files

1. Faster to Handle

- CSV files are lightweight.
- They can be processed quickly.

2. Smaller in Size

- CSV files store only text data.
- They take less storage space compared to many other file formats.

3. Easy to Generate and Import

- CSV files can easily be created using programs.
- They can also be imported into:
  - Spreadsheets
  - Databases
  - Applications

4. Human Readable

- People can open and understand CSV files easily.

- They can even be edited manually using a text editor.

5. Simple to Use

- CSV format is simple.
- Easy for programmers to implement and parse.

6. Supported by Many Applications

- Almost all software applications support CSV files.

Examples

- Microsoft Excel
- Google Sheets
- Database software
- Python programs

Simple Summary

Advantage	Meaning
Fast	Quick to process
Small Size	Uses less storage
Easy Import	Works with spreadsheets/databases

Human Readable	Easy for people to understand
Simple	Easy to create and use
Widely Supported	Works in many applications

Q15) Explain about CSV file operations in Python.

Ans:

CSV File Operations in Python

- Python has a built-in module called csv.
- This module is used to:
  - Read CSV files
  - Write CSV files

Importing CSV Module

Before working with CSV files, we must import the module.

Syntax

```
import csv
```

Why CSV Module is Used

- It correctly handles CSV files on all operating systems.
- Makes reading and writing tabular data easy.
- Helps work with rows and columns properly.

Basic Operations on CSV Files

There are mainly 2 operations:

1. Reading from a CSV file
2. Writing to a CSV file

Reading from a CSV File

- To read a CSV file:
    1. Open the file using open()
    2. Use csv.reader() to create a reader object

What is a Reader Object?

- A reader object reads the CSV file line by line.
- Each row is returned as a list.

Example CSV File (student.csv)

```
RollNo,Name,Marks
101,Akhil,95
102,Ravi,88
103,Anu,91
```

Program to Read CSV File

```
import csv

f = open("student.csv", "r")

reader = csv.reader(f)

for row in reader:
    print(row)

f.close()
```

Output

```
['RollNo', 'Name', 'Marks']
['101', 'Akhil', '95']
['102', 'Ravi', '88']
['103', 'Anu', '91']
```

Explanation

Step	Meaning
open()	Opens CSV file
csv.reader()	Converts file into reader object
for row in reader	Reads rows one by one
row	List containing field values

Using with open()

- with open() automatically closes the file after execution.
- No need to write close() manually.

Example Using with open()

```
import csv

with open("student.csv", "r") as f:

    reader = csv.reader(f)

    for row in reader:
```

```
    print(row)
```

Advantage of with open()

open()	with open()
Must close manually	Closes automatically
More chances of mistakes	Safer and cleaner

Important Points

- First row usually contains column names (header).
- Remaining rows contain actual records.
- Fields are separated using delimiters like:
  - Comma ,
  - Tab \t
  - Colon :
  - Semicolon ;

Simple Summary

Term	Meaning

csv module	Handles CSV files
reader()	Reads CSV data
Reader object	Reads rows as lists
with open()	Automatically closes file
